CS1540 Algorithmic design

David Mestel, Steven Chaplick

Homework 1.

1 Introduction

These exercises are simply for additional practice, but also include some implementation tasks that can be automatically tested via CodeGrade.

Namely, for the programming tasks ('The important machine' parts A and B), some tests are available via CodeGrade, which can be reached from the link on Canvas. Your program needs to read a filename from the terminal, read the input from the named file, and print the output to the terminal. The Canvas page for each task contains an example `.java` file demonstrating the I/O. Note that **the file you upload must have the same name and main class as the example, otherwise the testing will fail**. An example input file is also included, and its expected output.

2 The important machine (think Activity Selection)

Consider a single machine scheduling problem where we are given a set, T, of tasks specified by their start and finish times. All tasks run on only one, important, machine, and the important machine can only perform one task at a time. If a task cannot be scheduled at its assigned starting time it cannot be performed at all. We wish to maximize the number of tasks that this single machine performs.

The rules of scheduling are as follows ($[a, b]$ denotes the start and end times of a task):

- Times $[a, b]$ means the task must start at time step a (or not at all).
- Times $[a, b]$ means that the task ends before time step b starts. This agrees with the real-world convention for scheduling meetings (e.g., if a meeting is from 1-3, it starts at 1 o'clock and ends at 3 o'clock. It takes up the entire hours of 1 o'clock and 2 o'clock, and none of 3 o'clock).
- Therefore, the sequence $[a, b], [b, c]$ is allowable.

Note that in the input the tasks are not necessarily ordered according to start time, finish time, duration, or any other criteria. There may be multiple tasks with the same start and finish times.

(A) One greedy algorithm to schedule jobs on this machine is as follows:

- start from the first time slot and go chronologically
- if the machine is unused, schedule a task $t \in T$ that starts at that time slot (if one exists)
- if more than one task starts at that time slot, schedule the shortest task

Write a program that will accept a set of tasks and use the above algorithm to schedule them. Your program will output the tasks that the important machine can accomplish.

The first line of the input will be an integer $k < 10^7$. Each of the following k lines will consist of a pair of integers ab, with $a \leq b \leq 10^7$, the start and finish times of the kth task.

The output of your program should be similar: a sequence of lines, each containing a pair of integers ab, the start and finish times of a scheduled task. You must list the scheduled tasks in chronological order.

To test your program name it `MachineA.java` and submit it to Codegrade.

(B) The algorithm described in part 2 is not optimal. Your task here consists of three parts.

(i) Provide an example input that demonstrates that the algorithm in part 2 is not optimal.

(ii) Design and implement an algorithm which is guaranteed to always schedule the maximum possible number of jobs. The input will be in the same format as before, but now you should output only a single integer m, the maximum number of jobs that can be scheduled.

(iii) Prove that your algorithm always provides the optimal answer.

To test your program from (ii), name `MachineB.java`, and submit it to CodeGrade.

3 Divide and Conquer and Master Theorem exercises

For each question, briefly justify your answer.

1. Suppose we have recursive algorithms whose running times are given by each of the following definitions (together with $T(1) = 1$ in each case). Calculate the aysmptotic runtime for each.
   (a) $T(n) = 7T(n/3) + n^2$.
   (b) $T(n) = 3T(n/9) + \sqrt{n}$.
   (c) $T(n) = 4T(n/3) + n \log n$.
2. I am hoping to improve on Karatsuba's algorithm for integer multiplication, by spliting each n-bit integer into three parts rather than two. What is the largest number of multiplies (of $n/3$-bit integers) I can use in order to beat Karatsuba's algorithm (which uses 3 multiplies of $n/2$-bit integers)?
3. For which of the following does the Master Theorem apply? Where it does apply, calculate asymptotic runtime.
   (a) $T(n) = 3T(n/3) + n \log \log n$.
   (b) $T(n) = 8T(n/2) + n^3 \log n \log(n^2)$.